The Stela Governance Architecture: Establishing Statelessness via Procedural Scrubbing and Pre-Flight Validation

1. Introduction: The Entropy of the Living Surface

In the evolving discipline of repository-resident artificial intelligence, the primary challenge to long-term stability is not the capability of the underlying model, but the accumulation of "contextual sediment." As autonomous agents interact with a repository, they generate transient artifacts—logs, diffs, temporary checklists, and conversational residue—that, if left unchecked, calcify into a false historical record. This phenomenon poses a direct threat to the **Policy of Truth (PoT)**, the constitutional document governing the Stela ecosystem, which mandates a state of "Forever Now".[1]

The TASK.md file occupies a paradoxical position within this architecture. While the **Filing Doctrine** strictly segregates the repository into functional jurisdictions—ops/ for execution, docs/ for explanation, and projects/ for payload—TASK.md serves as the "Active Work Surface," a mutable interface where human intent meets machine execution.[1] This mutability creates a vulnerability: as the file accumulates specific session data (e.g., git hashes, branch names, and check-marked steps), it drifts from its structural purpose as a governance contract and becomes a "dirty" artifact. When a subsequent agent ingests this dirty state, it encounters "cognitive dissonance"—a conflict between the stale data in TASK.md and the actual state of the repository.[1]

To resolve this conflict and guarantee structural persistence without drift, the system must adopt a "metabolic" lifecycle. This report outlines the architectural transformation of TASK.md from a passive log into a strictly generic "Platonic Ideal"—a blank form that exists in a state of potentiality. This transformation relies on two engineered mechanisms: a metabolic cleaning engine (ops/bin/prune) that procedurally scrubs the surface back to its template state using advanced text processing logic, and a mandatory pre-flight immune system (tools/lint/dp.sh) that rejects any Dispatch Packet (DP) failing to adhere to the strict structural verification zones of the new schema. By enforcing a cycle of ingestion, digestion, and excretion, the Stela repository ensures that every worker session begins from a cryptographically verified, zero-state anchor, thereby eliminating the hallucination vectors inherent in static documentation.[1]

1.1 The Theoretical Framework: Metabolic Law and Statelessness

The concept of **Metabolic Law** posits that a repository must process work similarly to how a biological organism processes nutrients. A Dispatch Packet (DP) is "ingested" into the TASK.md surface. The agent "digests" this packet, performing work and generating outputs (code, documentation). Once the valuable nutrients (the completed work) are "absorbed" into the main branch via a Pull Request, the waste products—the specific instructions, temporary logs, and session-specific verifications—must be "excreted".[1]

Failure to excrete results in "sediment." In the context of Large Language Models (LLMs), sediment acts as "toxic context." If an agent sees a TASK.md that already contains a completed checklist from a previous session, it may hallucinate that the current work is also done, or it may conflate the previous session's Base HEAD with the current one, leading to operations on the wrong commit graph. Therefore, **Statelessness** is not merely a preference; it is a functional requirement for determinism. The TASK.md file must be returned to a mathematically predictable baseline—the Platonic Ideal—after every cycle.[3]

1.2 The Generation Mandate vs. The Checkbox Trap

A critical insight driving this reformation is the failure of the "checkbox" model. Previous iterations of TASK.md utilized pre-filled verification steps (e.g., - [ ] Verify tree). This design inadvertently encourages "lazy" agent behavior, where the model outputs a marked checkbox (- [x]) without actually executing the verification logic. This is a form of "compliance drift," where the symbol of compliance replaces the act of verification.[1]

The reformed architecture enforces a **Generation Mandate**. The Platonic Ideal of TASK.md removes all checkboxes and pre-filled responses. Instead, it provides *imperatives* (instructions) that compel the agent to generate fresh evidence. The agent cannot simply assert completion; it must produce a specific artifact (e.g., storage/handoff/DP-OPS-XXXX-RESULTS.md) containing the raw, unforgeable output of verification tools like tools/verify.sh.[1] This shifts the governance model from "trust" to "cryptographic proof," ensuring that every assertion of truth is backed by a generated receipt.[5]

---

2. The Platonic Ideal: Refining the TASK.md Schema

The refined TASK.md schema is designed as a self-correcting control plane divided into five interlocking zones. These zones serve as the immutable scaffolding of the session. While the content within these zones is transient, the *structure* of the zones is permanent and protected by the ops/bin/prune logic.

2.1 Zone 1: The Anchor (Session Integrity)

The Anchor serves as the cryptographic binding between the agent and the repository's current state. In the Platonic Ideal, this section must never contain hardcoded values (e.g.,

specific SHA hashes) in the template itself. Instead, it functions as a dynamic pointer, instructing the agent to fetch the truth from a trusted binary source.

Template Specification: Zone 1

STELA TASK DASHBOARD (LIVING SURFACE)

Status:

Owner:

Last Updated:

1. Session State (The Anchor)

Pointer: Session context output (generated by ops/bin/open)

Active Branch: (Must match session context output)

Base HEAD: (Must match session context output)

Context Manifest: ops/lib/manifests/CONTEXT.md (Checked by tools/lint/context.sh)

Structural Analysis

The phrase (Must match session context output) is a "cognitive forcing function." It prohibits the agent from assuming the state. The ops/bin/open binary generates a timestamped OPEN artifact containing the exact git hash at the moment of session start.[1] By forcing the agent to transcribe this hash into TASK.md during the session—and scrubbing it back to the placeholder instruction at the end—we ensure that the agent acknowledges the specific temporal reality of its work environment.[1] This mitigates "context drift" where an agent might otherwise hallucinate a relationship with a previous, unrelated commit.[6]

2.2 Zone 2: The Law (Logic Pointers)

This zone establishes the "Constitution" of the active session. To minimize the token cost of the context window, it utilizes a "Pointer-First" design. Rather than duplicating the text of the governance rules (which leads to version drift between TASK.md and PoT.md), this zone points strictly to the authoritative files.

Template Specification: Zone 2

2. Logic Pointers (The Law)

Primary Constraint: PoT.md (Policy of Truth).

2.1 Governance Pointers

Jurisdiction: PoT.md Section 3.

Git Authority: PoT.md Section 4.1.

Behavioral Standard: PoT.md Section 4.2.

Staffing Protocol: PoT.md Section 4.1.

2.2 Execution Pointers (The Toolchain)

Linguistic Precision: tools/lint/style.sh.

Structure Verification: tools/verify.sh.

Context Hygiene: tools/lint/context.sh.

Truth Integrity: tools/lint/truth.sh.

DP Validation: tools/lint/dp.sh.

llms Integrity: tools/lint/llms.sh.

Structural Analysis

This section acts as the "Definition of Done." By explicitly listing the execution toolchain (tools/lint/*.sh), the schema informs the agent of the policing mechanisms that will be applied to its output.[1] Ideally, the prune logic will preserve this text byte-for-byte, as it represents the immutable "Law" that does not change from task to task. The inclusion of tools/lint/dp.sh here is a recursive enforcement mechanism: the task acknowledges the existence of the very tool that validates the task.[1]

2.3 Zone 3: The Contract (Current Dispatch Packet)

This zone is the "Freshness Gate." It is the most dynamic section during execution but must be the most rigidly controlled during the pre-flight check. In the Platonic Ideal, this section is a blank form waiting to be instantiated by a Dispatch Packet.

Template Specification: Zone 3

3. Current Dispatch Packet (DP)

DP-XXXX: (Template)

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/dp-xxxx-template

Base HEAD: (Must match session context output)

Gate Artifacts (Must Match):

- OPEN: Base HEAD (Must match session context output)
- OPEN-PORCELAIN: Base HEAD (Must match session context output)
- Dump: Base HEAD (Must match session context output)
- Dump Manifest: Base HEAD (Must match session context output)

Preconditions:

- No commits on main.
- Working tree must be clean before execution begins.
- If Base HEAD changes, regenerate gate artifacts and update this gate before proceeding.
- Mandatory artifacts: OPEN, OPEN-PORCELAIN, dump payload, dump manifest, and RESULTS.md.

Gate Commands (Must Pass):

- bash tools/lint/context.sh

3.2 Required Context Load (Read Before Doing Anything)

Read these in order before making edits:

1. PoT.md
2. TASK.md
3. SoP.md
4. docs/MANUAL.md
5. docs/MAP.md

3.3 Scope and Safety

Objective: (One sentence goal)

In scope:

- (List exact boundaries)

Out of scope:

- (List explicit exclusions)

Safety and invariants:

- (List safety constraints)

Target Files allowlist (hard gate):

- (List exact files to touch)

3.4 Execution Plan (A-E Canon)

3.4.1 State (What is true now)

(Describe current state)

3.4.2 Request (What we are changing)

(Describe changes)

3.4.3 Changelog (Planned edits)

(List files and edits)

3.4.4 Patch / Diff (Implementation details)

(Detailed steps)

3.4.5 Receipt (Proofs to collect)

(List verification commands to capture)

Structural Analysis

The **Freshness Gate** is critical for preventing "Dirty State" violations.[1] By demanding that the OPEN artifact (session start) matches the Dump artifact (context snapshot) and the Base HEAD (git state), the schema forces a synchronization of all three timelines (Session, Context, Repo).[1] The Target Files allowlist is a "hard gate"; the tools/lint/dp.sh check will parse this list and verify that the agent does not touch any file not explicitly authorized here, enforcing the principle of Least Privilege.[1]

2.4 Zone 4: The Closeout (The Verification)

This zone implements the **Generation Mandate**. It replaces the checklist with a "Mandatory Closing Block" that consists solely of instructions for generating final artifacts.

Template Specification: Zone 4

4. Closeout (Mandatory)

- Execute docs/MANUAL.md Closeout Cycle in order (Verify, Harvest, Refresh, Log, Prune).
- Update SoP.md with a DP entry.
- Run Routing Closeout scrub:./ops/bin/prune --dp= --scrub.

Mandatory Closing Block

(Varied Wording provision: Each entry must use meaningfully distinct wording.)

Primary Commit Header (plaintext)

(Generate at closeout; do not pre-fill in TASK.md)

Pull Request Title (plaintext)

(Generate at closeout; do not pre-fill in TASK.md)

Pull Request Description (markdown)

(Generate at closeout; do not pre-fill in TASK.md)

Final Squash Stub (plaintext)

(Generate at closeout; do not pre-fill in TASK.md)

Extended Technical Manifest (plaintext)

(Generate at closeout; do not pre-fill in TASK.md)

Review Conversation Starter (markdown)

(Generate at closeout; do not pre-fill in TASK.md)

4.1 Thread Transition (Reset / Archive Rule)

- Append a THREAD END entry to the TASK.md Work Log at completion.
- Execute Routing Closeout scrub only after all receipts and SoP logging are complete.

Structural Analysis

The phrase (Generate at closeout; do not pre-fill...) acts as a sentinel instruction. During the prune process, the logic will look for these headers and strip away any content the agent added, reverting the line back to this exact instruction string.[1] This ensures that the next agent receives the instruction to generate, rather than a stale record of the previous agent's generation. This enforces "System 2" thinking (deliberate generation) over "System 1" thinking (copy-pasting).[1]

2.5 Zone 5: The Stream (Work Log)

The final zone is the only place where historical data accumulates *during* the session, but it must be mercilessly truncated at the end.

Template Specification: Zone 5

5. Work Log (Timestamped Continuity)

(No active thread)

Structural Analysis

The string (No active thread) is the "Clean State" marker. If tools/lint/dp.sh encounters this string, it knows the file is ready for a new DP. If it encounters a timestamped log entry, it flags the file as "Dirty" (unless the --test flag is active, indicating an ongoing session).[1]

---

3. Metabolic Engineering: The ops/bin/prune Logic

The ops/bin/prune binary is the physiological mechanism of the repository's metabolism. Its primary function is to perform a **schema-preserving semantic scrub**. Unlike a simple file reset (e.g., cp template.md TASK.md), prune must operate on the *existing* file to preserve any structural updates that might have been legitimately patched into the schema while stripping out the transient session data. This requires a robust state machine approach using awk, as sed is too brittle for context-aware block processing.[7]

3.1 Design Principles for Structural Persistence

1. **Idempotency:** Executing prune --scrub multiple times on the same file must result in the exact same "Platonic" output state without data loss or duplication.[9]
2. **Scaffolding Preservation:** The script must identify the immutable headers (e.g., ## 1. Session State) and preserve them, while neutralizing the values associated with them.
3. **Sanitization:** Dynamic values like Git hashes and Branch names must be replaced with generic placeholders (0000000, main) to prevent "Ghost Context".[1]
4. **Hygiene:** The binary must explicitly purge the storage/tmp directory to destroy any ephemeral artifacts (diffs, temp logs) that could leak information between sessions.[1]

3.2 Detailed Logic Specification (The awk State Machine)

The following awk logic defines the scrub_task function. It operates by reading TASK.md line-by-line and switching between processing modes (normal, in_closeout, skip_payload).

3.2.1 Variable Initialization

The script initializes baseline variables to overwrite session data:

- base_head = "0000000"
- branch = "main"
- pointer = "Session context output (generated by ops/bin/open)"

3.2.2 The Processing Loop

Zone 1 & 3 Sanitization (The Regex Replacements):

The script scans for specific keys and resets their values while preserving the keys.

- **Match:** /^### DP-/ -> **Action:** Replace with ### DP-XXXX: (Template).
- **Match:** /^[[:space:]]*Active Branch:/ -> **Action:** Replace value with main (Must match session context output).
- **Match:** /^[[:space:]]*Base HEAD:/ -> **Action:** Replace value with 0000000 (Must match session context output).
- **Match:** /^[[:space:]]*Pointer:/ -> **Action:** Replace value with the generic pointer string.
- **Match:** Gate Artifacts (OPEN:, Dump:) -> **Action:** Reset to : Base HEAD 0000000.

Zone 4 Reset (The Block Parser):

This is the most complex logic. The script enters a specific state when it encounters the Mandatory Closing Block.

- **Enter State:** /^[[:space:]]*Mandatory Closing Block/ sets in_closeout = 1.
- **Logic:** Inside this state, the script looks for sub-headers like Primary Commit Header. When found:
  1. Print the header.
  2. Print the *instruction* line: (Generate at closeout; do not pre-fill in TASK.md).
  3. Set skip_payload = 1. This flag suppresses the printing of all subsequent lines (the agent's actual commit message or PR description).
  4. The skip_payload flag is reset only when the *next* sub-header is found, allowing the script to "hop" over the filled content while keeping the structure.
- **Exit State:** Detection of ## 4.1 Thread Transition sets in_closeout = 0 and skip_payload = 0.

Zone 5 Truncation (The Tail Cut):

- **Match:** /^## 5\. Work Log/ -> **Action:** Print the header, print (No active thread), and immediately exit. This ensures that thousands of lines of work logs are instantly discarded, returning the file to its concise state.[1]

3.3 The storage/tmp Hygiene Logic

To enforce the "No External Artifacts" rule at the filesystem level, prune acts as a garbage collector.

Bash

```
clean_storage_tmp() {
  local tmp_dir="${REPO_ROOT}/storage/tmp"
  if [[ -d "$tmp_dir" ]]; then
    # Delete all files except.gitignore to maintain dir structure
    find "$tmp_dir" -mindepth 1 -not -name ".gitignore" -delete
    echo "storage/tmp scrubbed. Artifacts destroyed."
  fi
}
```

This ensures that if an agent dumped a temporary diff or a partial log into storage/tmp (an allowed action during execution), it is destroyed before the next agent can hallucinate based on it.[1]

3.4 Implementation: The ops/bin/prune Script

Bash

```
#!/usr/bin/env bash
set -euo pipefail

# ops/bin/prune - The Metabolic Engine
# Purpose: Scrub TASK.md to Platonic Ideal and clean storage/tmp

REPO_ROOT="$(cd "$(dirname "${BASH_SOURCE}")/../.." && pwd)"
TASK_FILE="${REPO_ROOT}/TASK.md"
TMP_DIR="${REPO_ROOT}/storage/tmp"
SCRUB=0

#... argument parsing for --scrub...

scrub_task() {
  local tmp_task
  tmp_task="$(mktemp "${TMP_DIR}/task-scrub.XXXXXX")"

  awk -v base_head="0000000" -v branch="main" -v pointer="Session context output (generated
```

```
by ops/bin/open)" '
  BEGIN { in_closeout=0; skip_payload=0 }

  function emit_instruction() {
    print "(Generate at closeout; do not pre-fill in TASK.md)"
    print ""
  }

  # 1. Truncate Work Log immediately upon finding the header
  /^## 5\. Work Log/ {
    print $0
    print "(No active thread)"
    exit
  }

  # 2. Sanitize DP Header
  /^### DP-/ { print "### DP-XXXX: (Template)"; next }

  # 3. Neutralize Session Anchors
  /^[[:space:]]*Pointer:/ { print "Pointer: " pointer; next }
  /^[[:space:]]*Active Branch:/ { print "Active Branch: " branch " (Must match session context output)";
next }
  /^[[:space:]]*Base HEAD:/ { print "Base HEAD: " base_head " (Must match session context output)";
next }

  # 4. Reset Gate Artifacts
  /^[[:space:]]*[-*][[:space:]]*OPEN:/ { print "- OPEN: Base HEAD " base_head; next }
  /^[[:space:]]*[-*][[:space:]]*Dump:/ { print "- Dump: Base HEAD " base_head; next }

  # 5. Handle Closeout Instructions (Preserve Headers, Wipe Content)
  /^[[:space:]]*Mandatory Closing Block/ { in_closeout=1; skip_payload=0; print; next }
  /^## 4\.1 Thread Transition/ { in_closeout=0; skip_payload=0; print; next }

  {
    if (in_closeout) {
      # Check for specific headers that define the closing block
      if ($0 ~ /^[[:space:]]*Primary Commit Header/) { print $0; emit_instruction(); skip_payload=1; next }
      if ($0 ~ /^[[:space:]]*Pull Request Title/) { print $0; emit_instruction(); skip_payload=1; next }
      if ($0 ~ /^[[:space:]]*Pull Request Description/) { print $0; emit_instruction(); skip_payload=1; next }
      if ($0 ~ /^[[:space:]]*Final Squash Stub/) { print $0; emit_instruction(); skip_payload=1; next }
      if ($0 ~ /^[[:space:]]*Extended Technical Manifest/) { print $0; emit_instruction(); skip_payload=1;
next }
      if ($0 ~ /^[[:space:]]*Review Conversation Starter/) { print $0; emit_instruction(); skip_payload=1;
next }

      # If we are skipping payload (content between headers), do not print
      if (skip_payload) { next }
```

```
    }
    # Default: print the line
    print
  }
' "${TASK_FILE}" > "${tmp_task}"

 mv "${tmp_task}" "${TASK_FILE}"
 echo "Scrubbed TASK.md to template baseline."
}

if (( SCRUB )); then
 scrub_task
 clean_storage_tmp
fi
```

---

4. The Immune System: tools/lint/dp.sh Logic

While prune handles the cleanup at the *end* of a task, tools/lint/dp.sh handles validation at the *start* (and during) the task. It acts as a mandatory pre-flight immune response that prevents a "malformed" Dispatch Packet from ever entering the metabolic cycle.

4.1 The Adversarial Validation Model

The dp.sh tool treats the agent as an untrusted actor. It assumes that the agent will try to introduce "Context Hazards"—references to external chat history, uploaded files, or non-existent paths—and actively blocks them. This validation is not a suggestion; it is a "Hard Gate".[1]

4.2 Detailed Validation Logic

The script must perform four distinct classes of checks to guarantee the integrity of the DP.

4.2.1 Structural Schema Validation

This check ensures that the DP strictly adheres to the 5-Zone Schema defined in Section 2. It verifies the presence and **strict ordering** of mandatory headers.

- **Logic:**
  1. Define an array of expected headers: ``.
  2. Scan the file to find the line number of each header.
  3. **Failure Condition:** If any header is missing, or if Line(Header[i]) > Line(Header[i+1]), exit with error. This enforces the narrative flow required for the agent's logic processing.[10]

4.2.2 The "No External Artifacts" Check (Context Hazard Linter)

Agents often try to reference artifacts that exist only in the LLM's conversation history (e.g., "As discussed," "See uploaded file," "Snippet 1"). This violates the Filing Doctrine, which states that truth exists only in the repo.

- **Regex Pattern Blacklist:**
  The script scans for case-insensitive matches of the following tokens:
  - uploaded file
  - paste
  - snippet above
  - chat history
  - previous prompt
  - as discussed
- **Failure Condition:** "FAIL: DP references external chat artifacts. Truth must reside in the repo (ops/, docs/, projects/).".[1]

4.2.3 The Generation Mandate Check (Placeholder Detection)

This check ensures that the agent has not lazily left template placeholders in the *Contract* section (Zone 3), while simultaneously ensuring it *has* left the instruction placeholders in the *Closeout* section (Zone 4).

- **Logic:**
  - **Zone 3 (Contract):** Scan for tokens like , , TBD, TODO (unless inside a code block). Presence is a FAIL.
  - **Zone 4 (Closeout):** Verify the presence of the literal string (Generate at closeout; do not pre-fill...). Absence implies the agent has pre-filled the data or deleted the instruction, which is a FAIL.[1]

4.2.4 The Existential Manifest Check

This is the most advanced check. It verifies that every file path mentioned in the Target Files allowlist actually exists in the repository or the dump manifest.

- **Logic:**
  1. Extract the Target Files allowlist block using sed or awk.
  2. Parse out file paths (lines starting with -).
  3. For each path:
     - Check if it exists on disk ([[-f "$path"]]).
     - If not, check if it is explicitly marked as (new).
  4. **Failure Condition:** "FAIL: Allowlist contains ghost file 'src/magic.py' which does not exist in the repository." This prevents the agent from hallucinating files that it *thinks* are there based on outdated training data.[1]

4.3 Implementation: The tools/lint/dp.sh Script

Bash

```
#!/usr/bin/env bash
set -euo pipefail

# tools/lint/dp.sh - The Immune System
# Usage:./tools/lint/dp.sh [path_to_dp_or_task_file]

TARGET="${1:-TASK.md}"
FAILURES=0

fail() { echo "FAIL: $*" >&2; FAILURES=1; }

# 1. Structural Schema Validation
declare -a EXPECTED_HEADERS=(
  "## 1. Session State (The Anchor)"
  "## 2. Logic Pointers (The Law)"
  "## 3. Current Dispatch Packet (DP)"
  "## 4. Closeout (Mandatory)"
  "## 5. Work Log (Timestamped Continuity)"
)

last_line=0
for header in "${EXPECTED_HEADERS[@]}"; do
  line_num=$(grep -nF "$header" "$TARGET" | cut -d: -f1 | head -n1)
  if [[ -z "$line_num" ]]; then
    fail "Missing schema section: '$header'"
  elif (( line_num <= last_line )); then
    fail "Section out of order: '$header' (Line $line_num) after Line $last_line"
  fi
  last_line="$line_num"
done

# 2. Context Hazard Detection
FORBIDDEN_TERMS=("uploaded file" "paste" "snippet above" "chat history" "previous prompt")
for term in "${FORBIDDEN_TERMS[@]}"; do
  if grep -iq "$term" "$TARGET"; then
    fail "Context Hazard detected: '$term'. References to chat artifacts are forbidden."
```

```
  fi
done

# 3. Existential Manifest Check (Allowlist)
# Extract lines between "Target Files allowlist" and the next header
# This uses sed to capture the block and grep to extract file paths
mapfile -t TARGET_FILES < <(sed -n '/Target Files allowlist/,/^#/p' "$TARGET" | grep -E '^- ' | sed 's/^- //')

for file in "${TARGET_FILES[@]}"; do
  # Remove potential comments or whitespace
  clean_file=$(echo "$file" | awk '{print $1}')

  # Allow explicit new files
  if [[ "$clean_file" == *"(new)"* ]]; then continue; fi

  if [[! -e "$clean_file" ]]; then
    fail "Ghost File in allowlist: '$clean_file' does not exist."
  fi
done

if (( FAILURES > 0 )); then
  echo "DP Validation FAILED."
  exit 1
fi

echo "OK: DP Validation Passed."
```

---

5. Integration and Migration Strategy

The transition to this hardened governance surface requires a precise sequence of operations to avoid disrupting active work.

5.1 Migration Phase 1: Toolchain Deployment

Before the TASK.md file itself is updated, the enforcement binaries must be in place.

1. **Deploy ops/bin/prune:** Implement the awk scrubbing logic. Verify it against a dummy TASK.md to ensure it correctly identifies and resets the headers without destroying the file.
2. **Deploy tools/lint/dp.sh:** Implement the pre-flight checks. Initially, run this in "permissive" mode (logging warnings instead of errors) to gauge the compliance of

current agents.

5.2 Migration Phase 2: The "Clean Slate" Refactor

Once tools are ready, the repository must undergo a "Reset Event."

1. **Commit TASK.md Reset:** The Operator must manually overwrite the existing TASK.md with the Platonic Ideal template defined in Section 2. This commit (e.g., DP-OPS-0048) establishes the new baseline.[1]
2. **Update PoT.md:** The Policy of Truth must be amended to explicitly reference the new "Metabolic Law" and "Generation Mandate," legalizing the aggressive scrubbing behavior of ops/bin/prune.[1]

5.3 Migration Phase 3: Operational Enforcement

1. **Pre-Flight:** Agents are instructed (via docs/MANUAL.md) that tools/lint/dp.sh is now a mandatory gate command.
2. **Post-Flight:** The ops/bin/prune --scrub command is added to the docs/MANUAL.md "Closeout Cycle," making it a standard operating procedure for every task completion.[1]

6. Conclusion

The architectural reformation of TASK.md represents a shift from "Governance by Trust" to "Governance by Architecture." By strictly defining the **Platonic Ideal** of the work surface and enforcing its persistence through the metabolic logic of ops/bin/prune and the immune response of tools/lint/dp.sh, we eliminate the accumulation of contextual sediment. This ensures that every agent—regardless of model generation or context window limitations—operates on a clean, cryptographically verified foundation, preserving the **Policy of Truth** and enabling the repository to scale its autonomy indefinitely without structural degradation. The system does not merely record its history; it metabolizes it, leaving behind only the pure potential of the next task.

Table 1: Comparison of Sedimented vs. Stateless Governance Models

Feature	Sedimented Model (Legacy)	Stateless Model (Reformed)
Session State	Hardcoded values (e.g., HEAD: a1b2c3d)	Dynamic Pointers (HEAD: (Must match session output))
Verification	Pre-filled checkboxes ([x]	Generation Mandate

	Verified)	(Imperatives to create receipts)
History	Accumulated work logs from previous DPs	Truncated logs ((No active thread)) via scrub
Closeout	Templates filled by the agent	Instructions restored by prune logic
Validation	Manual review of the text	Automated pre-flight lint/dp.sh checks
Context	"Dirty" (contains chat/transient artifacts)	"Clean" (Platonic Ideal + current DP only)

Table 2: Validation Failure States (tools/lint/dp.sh)

Check Type	Failure Condition	Error Message
Structure	Missing "3.1 Freshness Gate" header	FAIL: Missing schema section: '## 3.1 Freshness Gate'
Structure	"Closeout" appears before "Freshness Gate"	FAIL: Section out of order
Hazard	"See uploaded file" in text	FAIL: Context Hazard detected: 'uploaded file'
Manifest	Allowlist file src/ghost.py missing	FAIL: Ghost File in allowlist: 'src/ghost.py'
Mandate	Closeout instruction text deleted	FAIL: Generation Mandate violation: Instructions missing.

Works cited

1. Architectural Reformation of TASK.md_ Stateless Governance Surface

(Hardened).md
2. Supporting Markdown Search For LLMs - Build.ms, accessed February 11, 2026, https://build.ms/2026/2/2/supporting-markdown-search-for-llms/
3. Cutting Through the Noise: Smarter Context Management for LLM-Powered Agents, accessed February 11, 2026, https://blog.jetbrains.com/research/2025/12/efficient-context-management/
4. Context Engineering: How to Give LLMs the Right Information (and No More) - Medium, accessed February 11, 2026, https://medium.com/@noelmathew/context-engineering-how-to-give-llms-the-right-information-and-no-more-91fa08b7ed02
5. LLM Output Drift: Cross-Provider Validation & Mitigation for Financial Workflows - arXiv, accessed February 11, 2026, https://arxiv.org/html/2511.07585v1
6. Get my drift? Catching LLM Task Drift with Activation Deltas, accessed February 11, 2026, https://cispa.de/en/research/publications/84748-get-my-drift-catching-llm-task-drift-with-activation-deltas
7. Markdown Rendering with Awk - Graham Marlow, accessed February 11, 2026, https://mgmarlow.com/words/2024-03-23-markdown-awk/
8. bert-github/gf-markdown-awk: Awk library to convert markdown to HTML or to plain text, accessed February 11, 2026, https://github.com/bert-github/gf-markdown-awk
9. clean up the header of csv files with awk - Stack Overflow, accessed February 11, 2026, https://stackoverflow.com/questions/44349394/clean-up-the-header-of-csv-files-with-awk
10. Basic writing and formatting syntax - GitHub Docs, accessed February 11, 2026, https://docs.github.com/github/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax